

TECHNICAL DOCUMENTATION

VAGDEVI MALINENI

Project Name

Raritone – AI-Powered Virtual Try-On Platform

1. API DOCUMENTATION

Base URL

<http://localhost:5000/api>

Authentication APIs

Register User

POST /auth/register

Request:

```
{  
  "name": "John Doe",  
  "email": "john@gmail.com",  
  "password": "123456"  
}
```

Response:

```
{  
  "success": true,  
  "message": "User registered successfully"  
}
```

Login User

POST /auth/login

Request:

```
{  
  "email": "john@gmail.com",  
  "password": "123456"  
}
```

Response:

```
{  
  "success": true,  
  "token": "jwt_token"  
}
```

Product APIs

Get All Products

GET /products

Response:

```
{  
  "success": true,  
  "products": []  
}
```

Create Product

POST /products

Response:

```
{  
  "success": true,  
  "message": "Product created successfully"  
}
```

Update Product

PUT /products/:id

Delete Product

DELETE /products/:id

Avatar APIs

Upload Avatar

POST /avatar/upload

Response:

```
{  
  "success": true,  
  "avatarUrl": "cloudinary_url"  
}
```

Get Avatar

GET /avatar/:id

Measurement APIs

Add Measurements

POST /measurements

Request:

```
{  
  "height": 170,  
  "weight": 65,  
  "chest": 38,  
  "waist": 32  
}
```

Get Measurements

GET /measurements/:userId

Try-On APIs

Create Try-On Request

POST /tryon

Get Try-On Results

GET /tryon/:id

2. DATABASE SCHEMAS

User Schema

Field	Type
name	String
email	String
password	String
role	String

avatar	ObjectID
createAt	Date

Purpose

Stores user account information and authentication details.

Product Schema

Field	Type
name	String
description	String
category	String
Price	Number
stock	Number
Images	Array

Purpose

Stores product catalog information.

Measurement Schema

Field	Type
userid	ObjectID
chest	Number
waist	Number
hip	Number
shoulder	Numbe
height	Number
weight	Number

Purpose

Stores user body measurements.

Avatar Schema

Field	Type
userid	Objectid
avatarimage	string
gender	string
bodytype	string

Purpose

Stores avatar images and body information.

TryOn Schema

Field	Type
userid	objectid
Product id	objectid
avatarid	objectid
outputimage	string
status	string

Purpose

Stores virtual try-on sessions and generated outputs.

ProductMapping Schema

Field	Type
productid	ObjectId
avatarid	ObjectId
fitstore	Number
recommendedsize	String

Purpose

Stores AI-generated fitting and recommendation data.

3. ARCHITECTURE DIAGRAM

Frontend (React / Next.js)



Backend API (Node.js + Express)



Socket.IO
(Real-Time Communication)

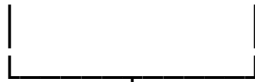


AI Service Layer
(Virtual Try-On Engine)



Cloudinary
(Media CDN)

MongoDB Atlas
(Database)



API Response



Frontend

4. SECURITY FEATURES

The backend implements multiple security layers to protect APIs, user data, and media resources.

JWT Authentication

- Secure login system
- Token-based authentication
- Protected routes
- Role-based authorization

Joi Validation

- Request validation
- Input sanitization
- Required field checks
- Data type verification

Helmet Security

Provides HTTP security headers:

- XSS protection
- Clickjacking prevention
- Content Security Policy

Morgan Logging

Tracks:

- Request method
- Status codes
- Response time
- Error monitoring

Rate Limiting

Prevents:

- Brute-force attacks
- API abuse
- Excessive requests

Example:

100 Requests / 15 Minutes

Error Middleware

Centralized error handling for:

- Validation errors
- Authentication errors
- Database errors
- Server errors

5. CLOUDINARY WORKFLOW

Cloudinary is used for cloud-based media storage and optimized image delivery.

Workflow

User Upload



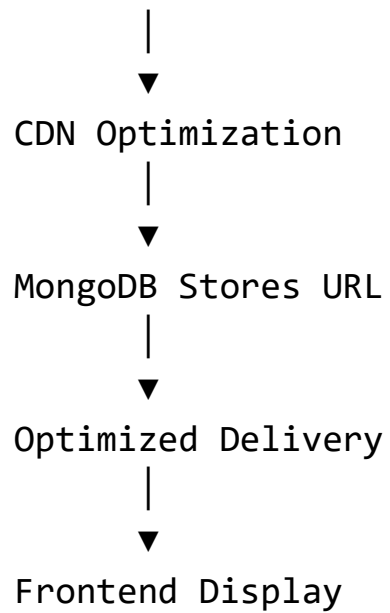
Multer Upload



Sharp Compression



Cloudinary Upload



Multer

Responsibilities:

- File upload handling
- File filtering
- Size validation

Sharp

Responsibilities:

- Image resizing
- Compression
- Format optimization

Cloudinary

Responsibilities:

- Cloud storage
- Image transformation

- Global CDN delivery

CDN Optimization

Benefits:

- Faster loading
- Lower latency
- Reduced server load

Cleanup Handling

- Temporary file deletion
- Failed upload cleanup
- Storage optimization

6. AI INTEGRATION PLAN

The AI layer will provide intelligent virtual try-on and recommendation capabilities.

Planned AI Workflow

User Avatar



Body Measurements



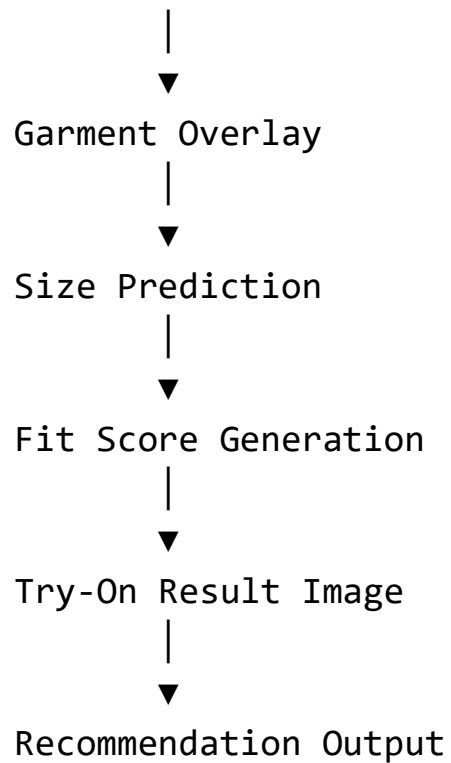
Body Analysis



Product Selection



AI Processing Engine



Planned AI Features

Virtual Try-On

- Clothing overlay generation
- Avatar-product matching
- Realistic garment fitting

Size Recommendation

- Personalized sizing
- Measurement-based predictions
- Fit confidence scoring

Recommendation Engine

- Product suggestions
- Personalized shopping experience
- User preference analysis

CONCLUSION

The Raritone backend architecture is designed to support a secure, scalable, and AI-ready virtual try-on platform. The system integrates Node.js, Express, MongoDB Atlas, Cloudinary, Socket.IO, and planned AI services to deliver real-time interactions, optimized media processing, and intelligent product recommendations. The architecture is cloud-ready, highly maintainable, and prepared for future scaling through caching, load balancing, microservices, and advanced AI integrations.